



Introduction to Artificial Intelligence (503043)

MIDTERM PRESENTATION

Group: 08

Instructor:

Nguyễn Thành An, MSc.

Team Members & Contribution Rates

	Student ID	Full Name	Academic Email	Assigned Responsibilities	Completion
1	524H0121	Lê Phú	524h0102@student.tdtu.edu.vn	Algorithm Core & Strategy (req 2,5,7)	34%
2	524H0095	Vũ Giang Huy	524h0095@student.tdtu.edu.vn	Modeling & Experiments(req 1,3,4)	33%
3	524H0102	Lê Hoàng Minh Khôi	524h0102@student.tdtu.edu.vn	Competitive Engine, GUI & Presentation (req 6,8, task2)	33%

Single-Agent Search & Non-Manhattan Heuristic

State S : (Agent, Boxes) with coordinates canonically sorted.

Actions $\mathcal{A}$:
{North, South, West, East}.

Transition: Unit step cost $c = 1$; boxes pushable if target cell is empty.

Goal Test: $\text{set}(\text{Boxes}) == \text{Goals}$

```
function PRECOMPUTE-MAZE-DISTANCES(goals, map) returns dist_table
    dist_table  $\leftarrow$  empty table
    for each g in goals do
        queue  $\leftarrow$  FIFO queue with g, dist_table[g, g]  $\leftarrow$  0
        while not EMPTY?(queue) do
            u  $\leftarrow$  POP-FIRST(queue)
            for each v in NEIGHBORS(u, map) do
                if v not in dist_table[g] then
                    dist_table[g, v]  $\leftarrow$  dist_table[g, u] + 1
                    queue  $\leftarrow$  APPEND(queue, v)
    return dist_table

function HEURISTIC(state, goals, dist_table) returns estimated_cost
    if IS-DEADLOCK(state) then return  $\infty$ 
    C  $\leftarrow$  cost matrix where C[i, j] = dist_table[goal_j, box_i]
    return MIN-WEIGHT-MATCHING(C)
```

Theoretical Proofs:

- **Admissibility** ($h \leq h^*$): Disregards dynamic box collisions and computes minimal bipartite assignment to goals, forming a strict lower bound of actual push steps.
- **Consistency** ($h(s) \leq c + h(s')$): Agent moves keep h constant ($\Delta h = 0 \leq 1$); a single box push decreases maze distance by at most 1 = $c(s, a, s')$. Triangle inequality holds.

A*Graph Search with Deadlock Pruning

```
function A-STAR-SEARCH(problem, h) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by  $f(n) = \text{PATH-COST} + h(n.\text{STATE})$ 
  frontier ← INSERT(node, frontier)
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if IS-DEADLOCK(child.STATE) then continue
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Key Implementation Highlights:

- **Graph-Search Integrity:** Uses explored set and frontier lookup to avoid redundant cyclical explorations.
- **Static Pruning:** IS-DEADLOCK eliminates non-goal corner and wall entrapments prior to frontier insertion, saving significant memory.

Empirical Evaluation: UCS vs. A*Search

=====					
EMPIRICAL EVALUATION: TIME AND SPACE COMPLEXITY					
=====					
Map	Algorithm	Time (s)	Nodes Expanded	Max Frontier	Cost

Easy	UCS	0.0004	56	15	8
	A*	0.0006	34	15	8
	Evaluation	Cost: PASS (Identical optimal cost)			A* prunes 1.6x nodes vs UCS

Medium	UCS	0.0049	698	138	13
	A*	0.0051	243	111	13
	Evaluation	Cost: PASS (Identical optimal cost)			A* prunes 2.9x nodes vs UCS

Hard	UCS	0.1026	10819	1891	18
	A*	0.0454	1741	859	18
	Evaluation	Cost: PASS (Identical optimal cost)			A* prunes 6.2x nodes vs UCS

Sample	UCS	0.4984	44111	10163	34
	A*	0.3492	10489	2546	34
	Evaluation	Cost: PASS (Identical optimal cost)			A* prunes 4.2x nodes vs UCS

- **Cost Optimality (C^* Match):** Both UCS and A* find identical optimal costs across all benchmarks (8,13,18,34 steps), proving heuristic admissibility.
- **Search Space Pruning:** A* drastically reduces expanded nodes as maze complexity scales (**1.6x** on Easy → **6.2x** on Hard → **4.2x** on Sample).
- **Execution Scalability:** On Sample map, A* reaches the target in **0.67s** (vs. 1.03s for UCS) while bounding frontier memory.

Competitive Two-Agent Engine & Decision Strategy

Simultaneous Execution: Both agents select actions simultaneously within a **1,000 ms** time constraint.

Collision Handling: Mutual movement into the same cell cancels displacement (both stand still).

Stealing & Ownership: Scored boxes belong to the scoring agent; pushing an opponent's box off goal resets it to neutral.

Step Limit: Terminated after n_{user} input steps; highest score wins.

```
function COMPETITIVE-AGENT(state) returns an action
  start_time ← CURRENT-TIME()

  if OPPONENT-BOX-ON-GOAL?(state) then
    path ← BFS-PATH(my_pos, steal_push_pos, obstacles)
    if path is not empty then return FIRST(path)

  if UNPLACED-BOXES?(state) then
    (box, goal) ← FIND-BEST-PAIR(unplaced_boxes, free_goals)
    path ← BFS-PATH(my_pos, push_pos, obstacles)
    if path is not empty then return FIRST(path)

  if (CURRENT-TIME() - start_time) < 0.8s then
    return ALPHA-BETA-SEARCH(state, depth = 2)

  return "Stay"
```

Strengths:

- Accurate geometric heuristic avoids dead ends without violating Manhattan/Euclidean restrictions.
- Modular multi-agent architecture cleanly separates game referee from agent decision scripts.
- Responsive Pygame GUI with complete playback control: Space (Pause/Resume), → (Forward), ← (Backward).

Trade-offs & Future Work:

- Hungarian matching complexity $O(n^3)$ introduces per-node calculation overhead on many boxes.
- Adversarial agent can be extended with Simultaneous Monte Carlo Tree Search (MCTS) for longer lookahead.